

CENTRO DE ENSEÑANZA TÉCNICA Y SUPERIOR



Escuela de Ingeniería

Machine Learning

Exercises Unit 2 C1

Teacher: Cesar Alan Torres Quiroz

Presented by:

Martin Alejandro Hermosillo Vargas - T028361

Tijuana, Baja California, September 17th, 2025

1.

EXERCISE 1 - CLASSIFY THE TASK (5 min)

For each, name the paradigm and subcategory:

- a) Predicting a student's GPA from study hours.
- b) Detecting whether an email is spam.
- c) Grouping customers by purchasing behavior.
- d) Predicting temperature from humidity and pressure.

a) Predicting a student's GPA from study hours.

- Paradigm: Supervised Learning
- Subcategory: Regression (the target variable, GPA, is continuous)

b) Detecting whether an email is spam.

- Paradigm: Supervised Learning
- Subcategory: Classification (the target variable is discrete/categorical: Spam vs. Not Spam)

c) Grouping customers by purchasing behavior.

- Paradigm: Unsupervised Learning
- Subcategory: Clustering (finding hidden patterns/groups in unlabeled data)

d) Predicting temperature from humidity and pressure.

- Paradigm: Supervised Learning
- Subcategory: Regression (the target variable, temperature, is continuous, similar to the linear regression code example)

2.

EXERCISE 2 - MANUAL OLS (10 min)

$$(x, y) = \{(1, 2), (2, 3), (3, 5), (4, 4), (5, 6)\}$$

Compute $\hat{\beta}_0$ and $\hat{\beta}_1$ by hand, using the worked example's table layout. Then verify with Python.

Check: $\bar{x} = 3$, $\bar{y} = 4$, and the slope comes out to 0.9.

i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
1	-2	-2	4.0	4
2	-1	-1	1.0	1
3	0	1	0.0	0
4	1	0	0.0	1
5	2	2	4.0	4
Σ			9.0	10

Slope ($\hat{\beta}_1$)

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{9}{10} = 0.9$$

Intercept ($\hat{\beta}_0$):

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} = 4 - (0.9 \cdot 3) = 4 - 2.7 = 1.3$$

Final regression Line : $\hat{y} = 1.3 + 0.9x$

```
[1] import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

plt.style.use("seaborn-v0_8-whitegrid")
✓ 56.6s Python
```

```
[2] x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2, 3, 5, 4, 6], dtype=float)

x_bar = x.mean()
y_bar = y.mean()

print(f"x-bar = {x_bar:.1f}")
print(f"y-bar = {y_bar:.1f}")
✓ 0.0s Python
```

... x-bar = 3.0
y-bar = 4.0

```
[3] centered = pd.DataFrame(
    {
        "x": x,
        "y": y,
        "x - x-bar": x - x_bar,
        "y - y-bar": y - y_bar,
    }
)

centered["centered product"] = centered["x - x-bar"] * centered["y - y-bar"]
centered["squared x deviation"] = centered["x - x-bar"] ** 2
centered
✓ 0.1s Python
```

	x	y	x - x-bar	y - y-bar	centered product	squared x deviation
0	1.0	2.0	-2.0	-2.0	4.0	4.0
1	2.0	3.0	-1.0	-1.0	1.0	1.0
2	3.0	5.0	0.0	1.0	0.0	0.0
3	4.0	4.0	1.0	0.0	0.0	1.0
4	5.0	6.0	2.0	2.0	4.0	4.0

```
numerator = centered["centered product"].sum()
denominator = centered["squared x deviation"].sum()

beta_1 = numerator / denominator
beta_0 = y_bar - beta_1 * x_bar

print(f"sum of centered products = {numerator:.1f}")
print(f"sum of squared x deviations = {denominator:.1f}")
print(f"beta_1 (slope) = {beta_1:.1f}")
print(f"beta_0 (intercept) = {beta_0:.1f}")
print(f"Fitted line: y-hat = {beta_0:.1f} + {beta_1:.1f}x")
```

✓ 0.0s Python

sum of centered products = 9.0
sum of squared x deviations = 10.0
beta_1 (slope) = 0.9
beta_0 (intercept) = 1.3
Fitted line: y-hat = 1.3 + 0.9x

```
X = x.reshape(-1, 1)
model = LinearRegression().fit(X, y)

print(f"scikit-learn Intercept (beta_0): {model.intercept_:.1f}")
print(f"scikit-learn Slope (beta_1): {model.coef_[0]:.1f}")
```

[5] ✓ 0.3s Python

... scikit-learn Intercept (beta_0): 1.3
scikit-learn Slope (beta_1): 0.9

3.

EXERCISE 3 - CODE & INTERPRET (15 min)

Modify the demo:

- Change the true slope to 3.0; does the model recover it?
- Increase the noise to $\sigma = 2.0$; watch R^2 drop.
- Reduce N to 10, what happens to the coefficient estimates?

(Connect (c) to Unit 1: fewer data $\rightarrow$ higher **variance** of the estimator.)

Part A For Part A, the model successfully recovers the true slope. With a sample size of $N = 50$ and a noise standard deviation of 0.5, the fitted slope obtained by OLS is 3.001, which is extremely close to the ground-truth value of 3.0. The fitted intercept reaches 2.024 compared to the true value of 2.0. Because the underlying signal dominates the relatively low noise, the model captures almost all variation in the data, yielding an R-squared value of 0.997.

Part B For Part B, increasing the noise standard deviation from 0.5 to 2.0 causes the R-squared value to drop significantly from 0.992 down to 0.887. Even though the estimated slope remains relatively accurate at 1.524 against the true slope of 1.5 due to maintaining $N = 50$, the larger error standard deviation increases the overall scatter of observations around the regression line. This higher noise elevates the residual sum of squares, thereby reducing the proportion of total variance explained by the model.

Part C For Part C, reducing the sample size to $N = 10$ causes the coefficient estimates to deviate noticeably from their true values, yielding a fitted slope of 1.445 (true value 1.5) and a fitted intercept of 2.464 (true value 2.0). This behavior connects directly to Unit I principles regarding sampling distribution: the variance of the slope estimator is inversely proportional to the sum of squared deviations of X . Having fewer data points decreases the denominator, which increases the estimator's variance. Consequently, while OLS remains an unbiased estimator on average, individual estimates from small samples become much more volatile and sensitive to random noise.

Base Code

```
[32] import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

Python
```

▶

```
[33] seed = 42
n = 50
true_intercept = 2.0
true_slope = 1.5
noise_std = 0.5

rng = np.random.default_rng(seed)

Python
```

```
[34] X = rng.uniform(0, 10, n).reshape(-1, 1)
y = true_intercept + true_slope * X.ravel() + rng.normal(0, noise_std, n)

Python
```

```
[35] model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)

print(f"Fitted intercept: {model.intercept_:.3f}")
print(f"Fitted slope: {model.coef_[0]:.3f}")
print(f"R-squared: {r2_score(y, y_pred):.3f}")

Python
```

```
... Fitted intercept: 1.884
Fitted slope: 1.506
R-squared: 0.992
```



Code with implemented changes

Part A

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```

[37] ✓ 0.0s Python

```
seed = 42
n = 50
true_intercept = 2.0
true_slope = 3
noise_std = 0.5

rng = np.random.default_rng(seed)
```

[38] ✓ 0.0s Python

```
X = rng.uniform(0, 10, n).reshape(-1, 1)
y = true_intercept + true_slope * X.ravel() + rng.normal(0, noise_std, n)
```

[40] ✓ 0.0s Python

```
model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)

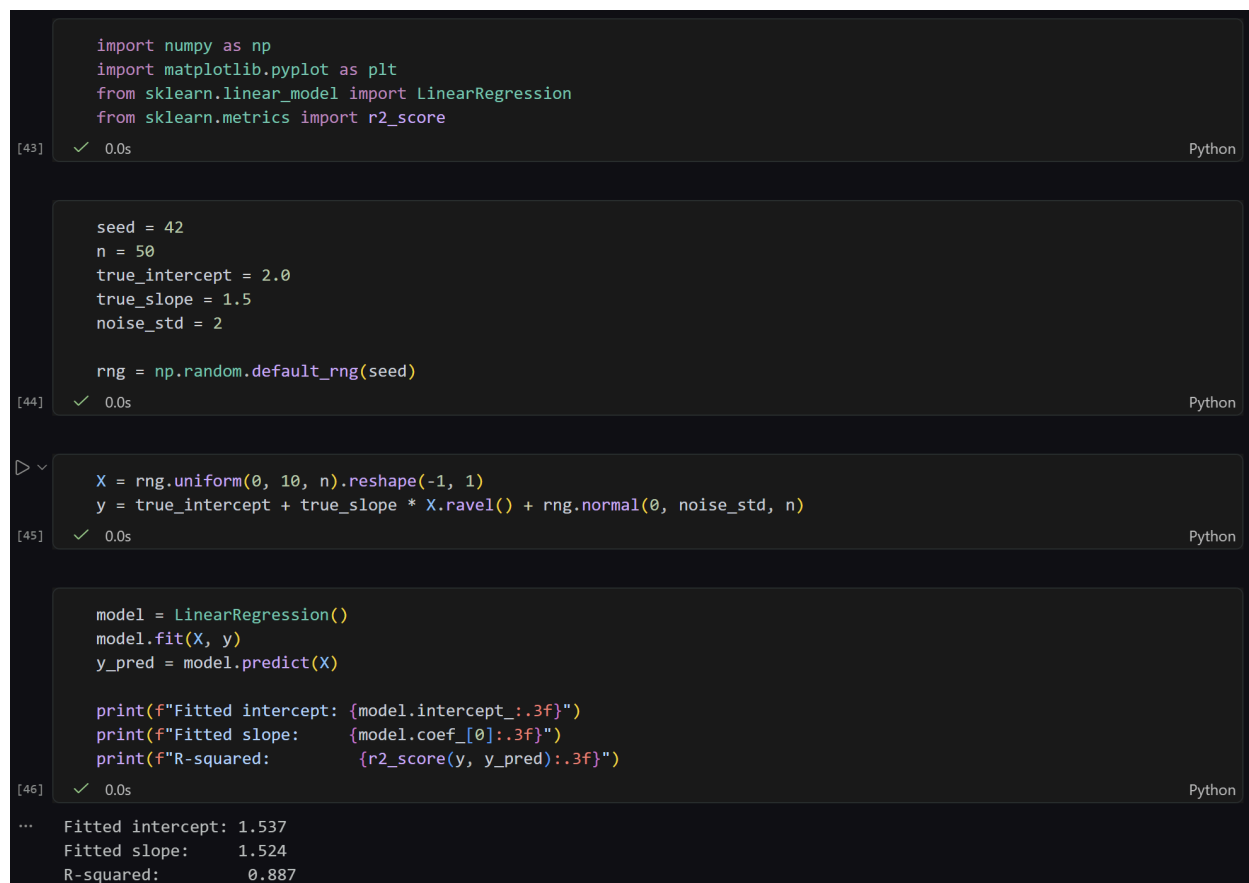
print(f"Fitted intercept: {model.intercept_:.3f}")
print(f"Fitted slope: {model.coef_[0]:.3f}")
print(f"R-squared: {r2_score(y, y_pred):.3f}")
```

[41] ✓ 0.0s Python

```
... Fitted intercept: 2.024
Fitted slope: 3.001
R-squared: 0.997
```



Part B





Part C

```
[48] import numpy as np
      import matplotlib.pyplot as plt
      from sklearn.linear_model import LinearRegression
      from sklearn.metrics import r2_score

      seed = 42
      n = 10
      true_intercept = 2.0
      true_slope = 1.5
      noise_std = 0.5

      rng = np.random.default_rng(seed)
```

Python

```
[49] X = rng.uniform(0, 10, n).reshape(-1, 1)
      y = true_intercept + true_slope * X.ravel() + rng.normal(0, noise_std, n)
```

Python

```
[50] model = LinearRegression()
      model.fit(X, y)
      y_pred = model.predict(X)

      print(f"Fitted intercept: {model.intercept_:.3f}")
      print(f"Fitted slope: {model.coef_[0]:.3f}")
      print(f"R-squared: {r2_score(y, y_pred):.3f}")
```

Python

```
[51] ... Fitted intercept: 2.464
      Fitted slope: 1.445
      R-squared: 0.995
```

```
X_sorted = np.sort(X, axis=0)
y_sorted_pred = model.predict(X_sorted)

plt.scatter(X, y, label="Observations")
plt.plot(X_sorted, y_sorted_pred, color="red", label="Fitted line")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

